

Somme de deux nombres

Écrire une fonction `somme(a, b)` qui retourne la somme des deux nombres **a** et **b** saisis par l'utilisateur.

Vous faites l'appel à la fonction.

Pair ou impair

Écrire une fonction `pair(n)` qui renvoie `True` si n est pair, sinon `False`.

Factorielle

Écrire une fonction `factorielle(n)` qui renvoie $n!$, où n est un entier positif ou nul.

le $n^{\text{ième}}$ terme d'une suite

On considère la suite définie par :

$$u_0 = 3, \quad u_{n+1} = 3u_n + 2.$$

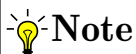
Écrire une fonction `suite(n)` qui renvoie le $n^{\text{ième}}$ terme de cette suite.

Nombre de diviseurs

Écrire une fonction `nombreDiviseurs(n)` qui prend en paramètres un entier **n** positif non nul. La fonction renvoie le nombre de diviseurs stricts de **n** qui sont supérieurs à 1.

***Nombre premier**

Écrire une fonction `estPremier(n)` qui prend en paramètres un entier **n** positif non nul et qui renvoie `True` si **n** est un nombre premier et `False` sinon.

**Note**

Un nombre est dit premier s'il n'admet aucun diviseur sauf 1 et le nombre lui-même ;
Exemple : 5, 7, 11, 13, 17

Somme de diviseurs

Écrire une fonction `sommeDiviseurs(n)` qui prend en paramètres un entier **n** positif non nul et qui retourne la somme de ses diviseurs stricts.

*****Nombre parfait**

Écrire une fonction `estParfait(n)` renvoie `True` si le nombre **n** est parfait et `False` sinon. Vous pouvez utiliser la fonction `sommeDiviseurs`.

Note

Un nombre est dit parfait s'il est égal à la somme de ses diviseurs stricts.

Exemple : 6, 28,

Vous proposez une solution en utilisant la boucle **for** et une autre en utilisant la boucle **while**.

Somme des nombres premiers

- ❶ Écrire une fonction *sommePremier()* qui prend en paramètres un nombre S et qui renvoie la **Somme** des premiers nombres premiers telle que **Somme** $\geq S$
- ❷ Faire un appel à la fonction par une valeur de votre choix

Exemple

$S = 15$ En sortie $\rightarrow 17$

$17 = 2 + 3 + 5 + 7$ avec 2, 3, 5, 7 sont des nombres premiers.